



Machine Learning

HOMEWORK 4

NAME:

STUDENT ID#:

- **Target Class Rule** - Each student uses a **personalized set of 3 classes** derived from their Student ID. Scan your Student ID **from right to left** and collect the first **3 distinct digits** you encounter. Use those digits as your `target_classes`.
- Example: Student ID 412540003 → Scan right-to-left: 3, 0, 0 (skip, duplicate), 0 (skip), 4 → `target_classes = [0, 3, 4]`
- Set this list in your code where indicated. All three tasks (VAE, GAN, DDPM) must use the **same** `target_classes`

Objectives:

- Understand and implement a **Variational Autoencoder (VAE)** using the reparameterization trick to enable gradient-based optimization over a stochastic latent space.
- Derive and minimize the **Evidence Lower Bound (ELBO)** loss, combining reconstruction error and KL divergence regularization.
- Understand and implement a **Generative Adversarial Network (GAN)**, training a generator and discriminator through an adversarial minimax objective.
- Analyze GAN training dynamics including discriminator/generator loss balance and sample quality.
- Understand and implement a **Denoising Diffusion Probabilistic Model (DDPM)**, including the fixed forward noising process and learned reverse denoising trajectory.
- Visualize generative outputs: VAE reconstructions and latent manifold, GAN sample grids, and DDPM iterative denoising sequences.

Part 1. Instruction

1. Variational Autoencoder (VAE)

In this task, you will implement a **Variational Autoencoder** trained on the **MNIST** dataset using **your 3 target classes** (determined by your Student ID (see the rule above)). The VAE consists of an encoder network that maps input images to a **Gaussian posterior** $q(z | x) = N(\mu, \sigma^2 I)$, and a decoder network that reconstructs images from sampled latent vectors. You must implement the **reparameterization trick** to allow gradients to flow through the stochastic sampling step. Train the model by minimizing the **ELBO loss** (reconstruction + β -weighted KL divergence). Generate two visualizations: (1) a reconstruction comparison of original vs. reconstructed images, and (2) a **2D latent space manifold grid** by sweeping the first two latent dimensions.

2. Generative Adversarial Network (GAN)

In this task, you will implement a **GAN** trained on the **FashionMNIST** dataset using **your 3 target classes** (same digits as Task 1). The GAN consists of a **Generator** that maps random latent noise vectors $z \sim N(0, I)$ to synthetic images, and a **Discriminator** that classifies images as real or fake. Both networks are trained alternately: the Discriminator is trained to maximize its ability to distinguish real from fake samples, and the Generator is trained to fool the



Discriminator. Use **Binary Cross-Entropy (BCE)** as the adversarial loss. After training, generate a 4×4 grid of synthesized FashionMNIST images.

3. Denoising Diffusion Probabilistic Model (DDPM)

In this task, you will implement a **DDPM** trained on the **MNIST** dataset using **your 3 target classes** (same digits as Task 1). The model uses a **fixed linear noise schedule** that progressively corrupts images over $T = 300$ timesteps. A **time-conditioned UNet** is trained to predict the noise added at each step. The forward diffusion process $q(x_t|x_0)$ is computed analytically in closed form. At inference time, images are generated by iteratively applying the **learned reverse denoising process**, starting from pure Gaussian noise. Visualize 6 snapshots across the denoising trajectory.

Part 2. Arithmetic Instructions

A. Variational Autoencoder (VAE)

Encoder maps input x to a posterior distribution over latent space:

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x) \cdot I)$$

Reparameterization Trick - samples z via a differentiable transformation:

$$z = \mu + \epsilon \cdot \sigma, \epsilon \sim \mathcal{N}(0, I), \sigma = \exp\left(\frac{1}{2} \log \sigma^2\right)$$

ELBO Loss — the objective to maximize (equivalently, minimize the negative):

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)]}_{\text{Reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z | x) \| p(z))}_{\text{KL. Regularization}}$$

Analytical KL Divergence (relative to standard Gaussian prior $p(z) = \mathcal{N}(0, I)$):

$$D_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Reconstruction Loss (MSE-based, summed over pixels):

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2$$

Total Loss per batch:

$$\mathcal{L}_{\text{VAE}} = \frac{1}{N} (\mathcal{L}_{\text{recon}} + \beta \cdot D_{\text{KL}})$$

B. Generative Adversarial Network (GAN)

Minimax Adversarial Objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))]$$

Discriminator Loss (maximize real = 1, fake = 0):



$$\mathcal{L}_D = -[\log D(x_{\text{real}}) + \log (1 - D(G(z)))]$$

Generator Loss (non-saturating formulation - maximize $\log D(G(z))$);

$$\mathcal{L}_G = -\log D(G(z))$$

Training Procedure (alternating updates):

Step	Action
1	Sample real images x from the dataset
2	Sample noise $z \sim \mathcal{N}(0, I)$; generate fake images $G(z)$
3	Update D to minimize $\mathcal{L}_D$ (freeze G)
4	Update G to minimize $\mathcal{L}_G$ using the updated D (freeze D)
5	Repeat until convergence

C. Denoising Diffusion Probabilistic Model (DDPM)

Noise Schedule (linear schedule over T steps):

$$\beta_t = \text{linspace}(10^{-4}, 0.02, T), \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

Forward Diffusion Process (closed-form, directly From x_0):

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Training Objective - predict the added noise ϵ using a time-conditioned network ϵ_θ :

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Reverse Denoising Step (learned posterior mean):

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) (t > 0)$$

where $\sigma_t = \sqrt{\beta_t}$.

Part 3. Setup Instructions

Step	Procedure
1	Clone the repository and navigate to the <code>hw4/</code> directory.
2	Install required libraries: <code>pip install torch torchvision matplotlib numpy</code>
3	Open colab.research.google.com or use your local Python environment.
4	The MNIST and FashionMNIST datasets are downloaded automatically via <code>torchvision.datasets</code> .
5	Run the provided <code>main.py</code> script. All output plots are saved to <code>./output_plots/</code> .
6	For Task 1 (VAE): Refer to the VAE class and <code>vae_loss</code> fn function.



7	For Task 2 (GAN): Refer to the Generator and Discriminator classes.
8	For Task 3 (DDPM): Refer to the SimpleDiffusionUNet and forward_diffusion function.

You may refer to the source code template we provide, expected output files:

File	Description
output_plots/vae_reconstructions.png	Original vs. VAE reconstructed MNIST images (2×8 grid)
output_plots/vae_manifold_grid.png	2D latent space manifold (10×10 grid sweep over dims 0 and 1)
output_plots/gan_generated_samples.png	GAN-synthesized FashionMNIST samples (4×4 grid)
output_plots/ddpm_denoising_timeline.png	DDPM reverse denoising snapshots (6 timestep frames)

Rules:

The dataset will be provided on E3, and sample code can be found here:

<https://github.com/ahsanfs/ML-Labs>

References:

Book References: Deep Learning: Foundations and Concepts 2024th Edition.

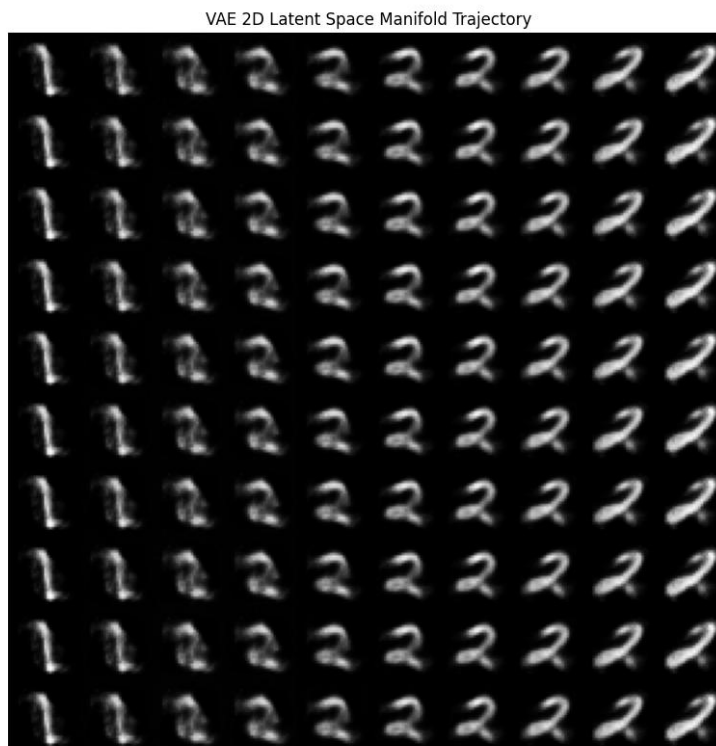
The lecture slide for an overview of Autoencoder, GAN, and Diffusion.

Example Output (Just for reference):

VAE Reconstructions

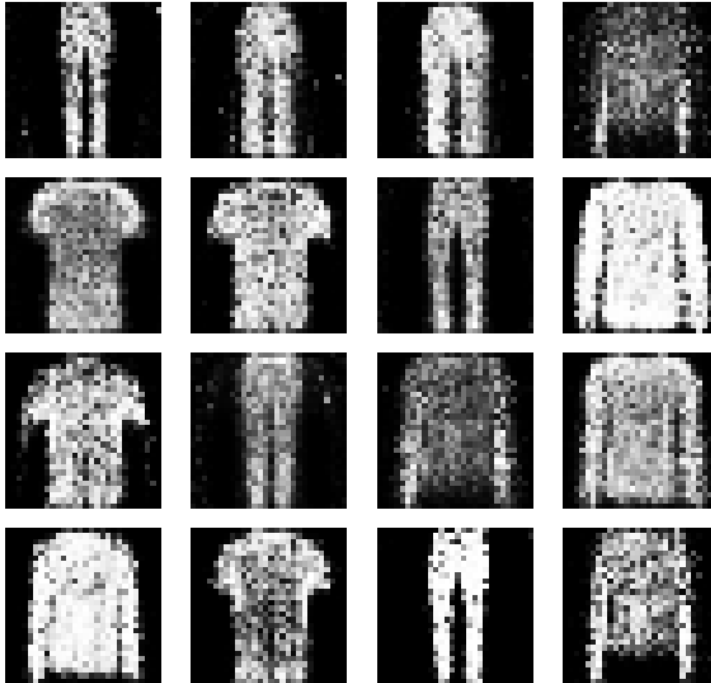


VAE Manifold Grid

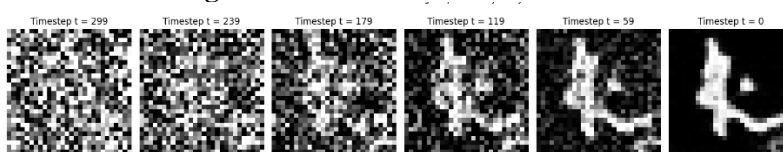


GAN Generated Samples

GAN Generated FashionMNIST Sample Grid



DDPM Denoising Timeline



Grading & Submission Instructions

Your report must present **results, figures, and analysis** for all three tasks. The report should follow the outline below.

1. (35%) Variational Autoencoder (VAE)

- 1.1. (5%) Correctly implement the VAE encoder outputting μ and $\log \sigma^2$ vectors, and the decoder reconstructing images from z .
- 1.2. (5%) Implement the reparameterization trick ($z = \mu + \epsilon \cdot \sigma$) to enable end-to-end gradient flow.
- 1.3. (5%) Implement the ELBO loss combining MSE reconstruction loss and analytical KL divergence.
- 1.4. (5%) Model trains correctly and ELBO loss decreases over epochs. Include a training loss curve or per-epoch printout.
- 1.5. Visualization:
 - 1.5.1. (5%) Plot the original vs. reconstructed images (2×8 grid). Describe the reconstruction quality.
 - 1.5.2. (5%) Plot the 2D latent space manifold grid (10×10). Describe the visual structure and interpolation behavior.
- 1.6. **Question:**
 - 1.6.1. (2.5%) What is the role of the KL divergence term in the ELBO? What would happen if you set $\beta = 0$ (no KL term).



- 1.6.2. **(2.5%)** How does the latent space manifold (2D grid) reflect the learned representation? Do you observe smooth interpolation between digit classes?

2. **(35%) Generative Adversarial Network (GAN)**

- 2.1. **(5%)** Correctly implement the Generator architecture mapping z - Fake image.
- 2.2. **(5%)** Correctly implement the Discriminator architecture classifying real vs. fake images.
- 2.3. **(5%)** Implement the alternating training loop: update D , then update G , in each iteration using BCE loss.
- 2.4. **(5%)** Model trains correctly. Include the $\mathcal{L}_D$ and $\mathcal{L}_G$ curves per epoch and discuss their behavior.
- 2.5. **(5%)** Plot the 4×4 grid of generated FashionMNIST samples and evaluate visual quality.
- 2.6. **Question:**
 - 2.6.1. **(5%)** Describe the training dynamics you observed. Did the discriminator and generator losses converge, oscillate, or collapse? What does this indicate about the training stability?
 - 2.6.2. **(5%)** Compare the visual quality of GAN-generated samples to VAE reconstructions. Which model produces more realistic-looking output? Which has more diversity? Why?

3. **(30%) Denoising Diffusion Probabilistic Model (DDPM)**

- 3.1. **(5%)** Correctly implement the forward diffusion $q(x_t | x_0)$ using the closed-form analytical expression with the precomputed α_t schedule.
- 3.2. **(5%)** Correctly implement the time-conditioned UNet and time embedding injection.
- 3.3. **(5%)** Train the model to predict noise ε using MSE loss. Include the per-epoch training loss and confirm it decreases.
- 3.4. **(10%)** Run the full reverse denoising inference loop from $x_T \sim N(0, I)$ down to x_0 . Plot the 6-frame denoising timeline showing the progression from noise to a recognizable digit.
- 3.5. **Question:**
 - 3.5.1. **(2.5%)** Describe what you observe in the denoising timeline. At which timestep does the image first become recognizable? What does this tell you about where the model "learns" the most?
 - 3.5.2. **(2.5%)** Compare DDPM image quality to VAE and GAN results. What are the tradeoffs between the three generative approaches in terms of sample quality, training stability, and inference speed?

Submission:

1. **Report:** The **report must be written in English**. Include screenshots of your results for each task (model training, evaluation, and plots).
2. **Code:** Submit your complete Python script (.py or .ipynb notebook).
3. Upload both your report and code to the E3 system. Name your files correctly as StudentID_HW4.zip with the following directory structure:

```

StudentID_HW4.zip |
├── 1. VAE
│   ├── Report: StudentID_report.pdf
│   └── Code: StudentID_code.py or StudentID_code.ipynb
├── 2. GAN
│   └── Report: StudentID_report.pdf
└──
```



└── Code: StudentID_code.py or StudentID_code.ipynb
└── 3. DDPM
└── Report: StudentID_report.pdf
└── Code: StudentID_code.py or StudentID_code.ipynb

4. Submission Rules:
 - 1 day late: **10% deduction** from the total score.
 - Incorrect file name: **10% deduction**.
 - Incorrect directory structure: **10% deduction**.
5. Plagiarism is **strictly prohibited**. Copying reports or code from **other students, previous submissions**, or **any external sources will result in penalties**.

